# COMPUTER GRAPHICS PROJECT REPORT

Norah hadi al saleem
443300164

Najran University

# INTRODUCTION

In this project, basic concepts from the computer graphics course were applied using the OpenGL library to create a three-dimensional model of a moving plane

The aircraft was divided into several main parts such as the body, wings, and tail, and was built using simple geometric shapes such as a cylinder, cone, and cube, with geometric transformations (rotation, translation, and scaling) applied to each part separately.

GLUT functions were also used to create the display window and continuously update the scene, allowing the aircraft to rotate around itself in a three-dimensional scene.

# THE CODE

```
 #include <GL/glut.h>


float angle = 0.0f;

void init() {
    glEnable(GL_DEPTH_TEST);
    glClearColor(0.7f, 0.9f, 1.0f, 1.0f);
}


void drawFuselage() {
    GLUquadric* quad = gluNewQuadric();

    glColor3f(0.9f, 0.1f, 0.1f);
    glPushMatrix();
    glRotatef(90, 0, 1, 0);
    gluCylinder(quad, 0.25, 0.25, 2.0, 32, 32);
    glPopMatrix();

    glColor3f(0.8f, 0.8f, 0.8f);
    glPushMatrix();
    glTranslatef(2.0, 0.0, 0.0);
    glRotatef(90, 0, 1, 0);
    glutSolidCone(0.25, 0.5, 32, 32);
    glPopMatrix();

    glColor3f(0.3f, 0.3f, 0.3f);
    glPushMatrix();
    glRotatef(90, 0, 1, 0);
    gluDisk(quad, 0.0, 0.25, 32, 1);
    glPopMatrix();
}
```

```
void drawWings() {
    glColor3f(0.2f, 0.2f, 0.2f);

    glPushMatrix();
    glTranslatef(0.5, 0.0, 0.6);
    glScalef(1.8, 0.06, 0.35);
    glutSolidCube(1.0);
    glPopMatrix();


    glPushMatrix();
    glTranslatef(0.5, 0.0, -0.6);
    glScalef(1.8, 0.06, 0.35);
    glutSolidCube(1.0);
    glPopMatrix();
}

void drawTail() {
    glColor3f(0.2f, 0.2f, 0.2f);

    glPushMatrix();
    glTranslatef(0.1, 0.0, 0.35);
    glScalef(0.6, 0.04, 0.2);
    glutSolidCube(1.0);
    glPopMatrix();

    glPushMatrix();
    glTranslatef(0.1, 0.0, -0.35);
    glScalef(0.6, 0.04, 0.2);
    glutSolidCube(1.0);
    glPopMatrix();

    glPushMatrix();
    glTranslatef(0.1, 0.3, 0.0);
    glScalef(0.3, 0.5, 0.04);
    glutSolidCube(1.0);
    glPopMatrix();
}
```

```
void drawPlane() {
    drawFuselage();
    drawWings();
    drawTail();
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();

    gluLookAt(4.0, 2.5, 5.0,
              1.0, 0.0, 0.0,
              0.0, 1.0, 0.0);

    glRotatef(angle, 0.0, 1.0, 0.0);
    drawPlane();

    glutSwapBuffers();
}

void timer(int value) {
    angle += 0.5f;
    if (angle > 360.0f) angle -= 360.0f;
    glutPostRedisplay();
    glutTimerFunc(16, timer, 0);
}

void reshape(int w, int h) {
    if (h == 0) h = 1;
    float aspect = (float)w / h;

    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45.0, aspect, 1.0, 100.0);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}
```

```
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB |
GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("3D Airplane");

    init();
    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutTimerFunc(0, timer, 0);

    glutMainLoop();
    return 0;
}
```

# OUTPUTS